

BANK AI • TECHNICAL WALKTHROUGH

From RAG to Tool Calling

How an LLM moves from external knowledge to external capabilities



Our Bank AI Assistant Architecture

The LLM decides whether a tool is needed. The application executes it. All tool results return to the LLM, which produces the final answer.



User

User question



LLM

Decides whether a tool is needed



Available tools

- **search_bank_policy()** — Uploaded-PDF retrieval with FAISS
- **get_account_balance()** — Mock banking API
- **get_recent_transactions()** — Mock banking API

BOUNDARY

LLM requests a tool. The application executes it.

LEARNING-ONLY

Banking APIs are mock tools, not real banking systems.

Step 1 — Define Tools

A tool definition gives the model a name, description, parameters, and implementation.

PYTHON · TOOL SURFACE

```
def get_account_balance(account_id: str) -> float:
    """Get the account balance for a given account ID.
    Returns the balance as a float value.
    """
    return 1234.56

def search_bank_policy(policy_id: str) -> str:
    """Search for a specific bank policy by ID.
    Returns the policy text as a string.
    """
    return "Policy text for ID: " + policy_id
```

Policy path

question →
vectorstore.similarity_search(k=3) →
relevant policy text

DESCRIPTION = ROUTING LOGIC

Balance questions route to
get_account_balance; policy questions
route to **search_bank_policy**.

Step 2 — LLM Tool Selection

Bind the tools once. Let intent and arguments route the request.

```
llm_with_tools = llm.bind_tools(tools)
```



INTENT

Complaint acknowledgement
timing

search_bank_policy



INTENT

Current balance

get_account_balance



INTENT + ARGUMENT

Last 3 transactions

get_recent_transactions
number_of_transactions = 3

LLM chooses the tool and generates arguments.

Application executes the selected tool.

Step 3 — Tool Call → Tool Result → Final Answer

User → LLM decides → Tool Call → Application executes tool → Tool Result returns data → LLM writes the Final Answer.

BALANCE EXAMPLE

```
get_account_balance(account_id="12345")
```

Mock API returns **£85,420.50**

"Your current account balance is £85,420.50."

COMPACT LOOP

```
response = llm_with_tools.invoke(messages)
for call in response.tool_calls:
    tool = tools_by_name[call["name"]]
    result = tool.invoke(call["args"])
    messages.append(ToolMessage(result,
                                tool_call_id=call["id"]))
```

THE NEXT STEP

From Tool Calling to Agentic AI

Tool calling lets an LLM request capabilities; an agent can dynamically decide what actions to take across multiple steps.

Simple LLM

Answers from the prompt

RAG

Accesses knowledge

Tool Calling

Requests capabilities

Agent

Chooses actions across steps

LEARNING EXAMPLE

"Check my balance and recent transactions and tell me whether I have enough money to cover my recent electricity payment."

Agent calls balance → observes result → calls transactions → synthesizes an answer.

THREE PART TAKEAWAY

RAG = knowledge

Tool Calling = capabilities

Agents = deciding how to use capabilities to accomplish a goal